

Lemuel 프로젝트에 사용된 알고리즘

이커머스 + 정산 MSA 플랫폼에서 **실제 구현된** 알고리즘을 코드 근거와 함께 정리한 문서. 라이브러리에 위임한 표준 알고리즘(BCrypt, JWT 등)과 직접 구현한 도메인 알고리즘을 구분한다.

조사 근거: 정산/보류 도메인, PG 대사 매치, Slug 생성기, Outbox claim 리포지토리, Rate Limit 레지스트리 등 핵심 클래스 정독.

1) 정산금 계산 (Settlement Calculation)

위치: `settlement-service/.../settlement/domain/Settlement.java`

```
commission = paymentAmount × commissionRate    (HALF_UP, scale 2)
netAmount  = paymentAmount - commission         (HALF_UP, scale 2)
```

- `BigDecimal + RoundingMode.HALF_UP, scale 2` 로 **금액 반올림 오차 제거** (부동소수점 미사용).
- 수수료율은 차등: 기본 3%(`COMMISSION_RATE`), 셀러 티어별 `SellerTier.rate()` 주입. 적용 `rate` 를 `commissionRate` 컬럼에 **이력 보존**.
- **환불 반영 재계산**: `netAmount = (paymentAmount - refundedAmount) - commission`. 누적 환불액이 결제액 초과 시 예외, `net ≤ 0` 이면 자동 `CANCELED`. `DONE` 정산은 불변 → `SettlementAdjustment` 별도 레코드로만 상쇄(원장 정합성).

2) 홀드백 + 영업일 계산 (Holdback & Business-Day)

위치: `settlement/domain/HoldbackPolicy.java`,
`Settlement.applyHoldback/consumeHoldbackForRefund`

- 보류액: `holdbackAmount = netAmount × holdbackRate` (`HALF_UP`). 즉시 지급액 = `netAmount - holdbackAmount`.
- 등급별 정책(`switch`): `NORMAL 30%/30일`, `VIP 10%/14일`, `STRATEGIC 0%(즉시)`.
- **영업일 가산 알고리즘**(`BusinessDayCalculator.addBusinessDays`): 정산일 + N 영업일로 release 예정일 계산 (주말 등 비영업일 skip).
- **환불 시 holdback 우선 차감**: `consumed = min(holdbackAmount, refundAmount)` — 보류분으로 환불을 먼저 흡수하는 그리디 차감. 잔액 0 이면 자동 해제.

3) PG 대사 매칭 (Reconciliation Matching) ★

위치: `pgreconciliation/domain/PgReconciliationMatcher.java`

PG 정산파일 vs 내부 결제원장 양방향 비교. 순수 도메인 로직(프레임워크 의존 0).

- **해시 인덱싱**으로 $O(n^2) \rightarrow O(n+m)$:
 - `Map<String,Integer>.merge(key,1,Integer::sum)` → 중복 거래 카운트
 - `Map<String,Row>.putIfAbsent` → `pg_transaction_id` 키 인덱스

- `Set<String>` → 중복 키 집합
- 양방향 스캔: (PG→내부) 절반 + (내부→PG) 절반으로 누락 양방향 검출.
- 불일치 분류: `DUPLICATE` / `MISSING_INTERNAL` / `MISSING_PG` / `ROUNDING_DIFF`(차이 < 1원, 자동보정) / `AMOUNT_MISMATCH`(≥1원, 운영자 검토).
- 임계값 `ROUNDING_THRESHOLD = 1.00` 으로 반올림 차이와 실제 오차 분리.

4) 한글 Slug 생성 (Hangul Romanization) ★

위치: `category/util/SlugGenerator.java`

한글 카테고리명 → URL-safe slug 변환. 한글 음절 분해 알고리즘 직접 구현.

```

유니코드 = ch - 0xAC00
초성 = 유니코드 / (21 × 28)
중성 = (유니코드 % (21 × 28)) / 28
종성 = 유니코드 % 28

```

- 한글 음절(U+AC00~U+D7A3)을 초성(19)·중성(21)·종성(28) 인덱스로 분해 → 로마자 테이블 매핑.
- 전처리: `Normalizer.Form.NFD` 정규화 + `\p{M}`(발음 구별 기호) 제거, 공백/특수문자 → 하이픈, 연속 하이픈 축약, 앞뒤 하이픈 제거.
- 계층형 slug: `parentSlug + "-" + childSlug`.

5) 분산 작업 큐 — Outbox Claim (FOR UPDATE SKIP LOCKED) ★

위치: `shared-common/.../outbox/...SpringDataOutboxEventRepository, ClaimOutboxEventPort`

다중 워커가 경합 없이 outbox 행을 나눠 발행하는 **distributed work-queue** 알고리즘.

```

SELECT id FROM outbox_events
WHERE status = 'PENDING'
  AND (claimed_at IS NULL OR claimed_at < now() - lease)
ORDER BY created_at
LIMIT :limit
FOR UPDATE SKIP LOCKED

```

- `FOR UPDATE SKIP LOCKED` → 다른 워커가 잠근 행은 건너뛴(**disjoint 분할**), 동시 폴링 시 중복 발행 0.
- 리스(lease) 기반 회수: claim 후 `claimed_at` 스탬프 → 워커가 죽어도 리스 만료 후 다른 워커가 재claim (at-least-once 보장).
- 발행 실패 시 `releaseClaim` 으로 즉시 재클레임 가능. `created_at` 오름차순 → **순서 보존(FIFO)**.

6) Rate Limiting — Token Bucket

위치: `shared-common/.../ratelimit/RateLimitRegistry.java` (Bucket4j)

- `ConcurrentHashMap.computeIfAbsent` 로 (정책|키) 별 버킷 lazy 생성(스레드 안전).

- **Token Bucket:** `capacity + refillGreedy(capacity, window)` — 윈도우 동안 균등 보충, 버스트 허용.

7) 멱등성 알고리즘 (Idempotency)

- **이벤트 멱등 3단 방어:** `outbox_events.event_id`(UUID UNIQUE) → `processed_events`(consumer_group+event_id 복합 PK) → `settlements.payment_id`(UNIQUE). 중복 이벤트가 어느 단계에서든 차단됨 (exactly-once 효과).
- **환불 멱등:** `refunds.idempotency_key` + 비관적 락(Pessimistic Lock)으로 동시 환불 요청 중복 처리 방지.

8) 동시성 제어 (Concurrency Control)

- **낙관적 락(Optimistic Lock):** `settlements.version, payouts.version(@Version)` — 동시 확정/지급 충돌 시 재시도.
- **비관적 락(Pessimistic Lock):** 환불 처리 시 결제 행 잠금.
- **분산 스케줄러 락(ShedLock):** `SchedulingLockConfig` — 다중 인스턴스에서 `@Scheduled` 배치가 한 노드에서만 실행되도록 보장.

9) PG 라우팅 + Fallback (Failover)

위치: `payment/adapter/out/pg/PgRouter.java`

- `LinkedHashMap` 순서 = 우선순위. 선택 알고리즘: **고객 거래 우선 PG** → **결제수단별 1순위** → **fallback** 체인 순회.
- 후보가 unhealthy/미지원이면 다음 후보로 자동 전환(failover). 거래 ID prefix 로 원처리 PG 역추적해 capture/refund 라우팅.

10) 회복탄력성 패턴 (Resilience4j)

위치: TossPG 연동 어댑터, PG 어댑터

- **Circuit Breaker:** PG 장애 시 회로 차단 → fast-fail.
- **Retry + Exponential Backoff:** 일시 오류 재시도(지수 백오프).
- **Bulkhead:** PG별 동시 호출 격리(자원 고갈 전파 차단).
- **Kafka DLT + 재처리:** `KafkaErrorHandlerConfig`(백오프 재시도 후 DLT), `DlqReplayService`(재처리).

11) 복식부기 정합성 (Double-Entry Invariant)

위치: `ledger/...` (`SingleLedgerEntryWriter`, `ReverseEntryService`)

- 모든 거래를 차변(`debit_account`)/대변(`credit_account`) 쌍으로 기록 → 회계 항등식(차변 합 = 대변 합) 유지.
- 정정은 삭제가 아닌 **역분개(reverse entry)**: `PENDING` → `POSTED` → `REVERSED`. 일별 합계 대사 (`ReconcileDailyTotalsService`)로 불변식 검증.

12) 라이브러리 위임 표준 알고리즘

알고리즘	용도	위치
BCrypt (cost=12)	비밀번호 해싱 (salt + adaptive)	PasswordHasherAdapter
JWT HS256 (HMAC-SHA256)	토큰 서명/검증	JwtUtil
Spring Batch chunk	청크 단위 정산 배치 처리	SettlementScheduler 등
Caffeine (W-TinyLFU)	L1 캐시 제거 정책	CacheConfig

정리

- **직접 구현 핵심 알고리즘**: 해시 인덱스 양방향 대사 매칭, 한글 음절 분해 로마자 Slug, FOR UPDATE SKIP LOCKED 분산 작업 큐, 영업일 가산, 홀드백 그리디 차감.
- **금액 처리**: 전 구간 BigDecimal + HALF_UP scale 2 (정확성 우선).
- **동시성/먹등**: 낙관·비관 락 + ShedLock + 3단 먹등 방어 + Idempotency-Key.
- **회복탄력성**: Circuit Breaker / Retry+Backoff / Bulkhead / Kafka DLT.
- **표준 알고리즘**: BCrypt, JWT HS256, Token Bucket, W-TinyLFU 캐시는 검증된 라이브러리에 위임.

부록) 코딩테스트 빈출 알고리즘 — 중요도 순 분석

위 본문이 "실무 도메인 알고리즘"이라면, 이 부록은 코딩테스트(코테)에 나오는 알고리즘을 중요도(빈도×ROI)순 으로 분석한다. 자료구조 관점은 자료구조.md 부록2와 함께 보면 된다.

중요도 = 출제 빈도 × 학습 ROI. 빈도 높고 투자 대비 점수 효율이 좋은 순. 난이도(★)는 코테 기준 체감.

Tier S — 필수 (안 풀면 광탈, 거의 모든 코테)

알고리즘	빈도	난이도	핵심 신호	복잡도
구현 / 시뮬레이션	★★★★★	★★☆	"시키는 대로 정확히" (지도 이동, 시계, 게임 규칙)	문제마다
완전탐색(브루트포스)+백트래킹	★★★★★	★★☆	n 작음(≤20), "모든 경우/조합/순열"	O(2^n), O(n!)
BFS / DFS (그래프·그리드)	★★★★★	★★☆	"최단(가중치 X)/영역/연결/미로"	O(V+E)
정렬 + 이분탐색	★★★★★	★★☆	"정렬 후", "조건 만족 최소/최대 값"	O(n log n)
그리디(Greedy)	★★★★☆	★★★	"매 순간 최적 선택", 회의실·동전·구간	O(n log n)
DP(동적계획법)	★★★★☆	★★★★	"경우의 수/최대최소, 겹치는 부분문제"	상태수×전이

합격선의 핵심: 구현·BFS/DFS·이분탐색·그리디만 안정적으로 풀어도 대부분 코테 통과선. **DP가 변별력**(당락을 가른다).

S-1. 구현/시뮬레이션

- 알고리즘이라기보다 **정확한 코딩력**. 삼성 SW 역량테스트의 절반.

- 팁: 방향 배열 dx/dy , 좌표 회전, 상태를 객체/배열로. 실수 줄이는 *차분한 구현*이 실력.

S-2. 완전탐색 + 백트래킹

- n 이 작으면($\leq 20 \sim 25$) 일단 완전탐색 의심. **백트래킹**은 가지치기(pruning)로 불필요한 분기 컷.
- 순열/조합/부분집합 → 재귀 + **visited**, 또는 비트마스크.

S-3. BFS / DFS

- **BFS**=최단거리(간선 가중치 1)·레벨(큐), **DFS**=완전탐색·연결성·사이클(재귀/스택).
- 그리드 문제는 2D 배열이 곧 그래프(상하좌우). 섬 개수, 미로 최단, 영역 채우기(flood fill).

S-4. 정렬 + 이분탐색

- 정렬은 보통 라이브러리(**Arrays.sort** $O(n \log n)$). 직접 구현(머지/퀵/힙)은 원리 이해용.
- **이분탐색**: 정렬된 데이터에서 $O(\log n)$. **파라메트릭 서치**("조건을 만족하는 최소/최대?")가 코테 단골 — 답을 이분.

S-5. 그리디

- 지역 최적 = 전역 최적임을 **증명(교환 논법)** 할 수 있어야 안전. 반례에 취약하니 주의.
- 회의실 배정(끝나는 시간 정렬), 거스름돈, 최소 회의 등.

S-6. DP

- **점화식 + 메모이제이션**(Top-down 재귀+캐시 / Bottom-up 테이블).
- 빈출 유형: 배낭(knapsack), LIS(최장증가부분수열), LCS, 동전, 계단, 구간 DP, 비트마스크 DP.
- 핵심: **상태 정의** → **전이** → **기저**. 상태 수 $\times$ 전이 비용 = 복잡도.

Tier A — 자주 (중급, 변별 구간)

알고리즘	빈도	난이도	핵심 신호	복잡도
투포인터 / 슬라이딩 윈도우	★★★★☆	★★☆	"정렬된 배열의 쌍/연속 구간 합·길이"	$O(n)$
누적합(Prefix Sum)/차분	★★★★☆	★★☆	"구간 합을 여러 번 질의"	전처리 $O(n)$, 질의 $O(1)$
다익스트라	★★★★☆	★★★	"가중치 ≥ 0 최단경로"	$O(E \log V)$
유니온-파인드(DSU)	★★★★☆	★★★	"같은 그룹?/연결/사이클"	$\sim O(\alpha(n))$
위상정렬	★★★★☆	★★★	"선후관계/의존성 순서"	$O(V+E)$

A 티어부터는 "자료구조 + 알고리즘" 결합. 다익스트라=그래프+힙, DSU=서로소집합, 누적합=배열 전처리.

Tier B — 가끔 (중상, 특정 시험·고득점)

알고리즘	빈도	난이도	핵심 신호
MST(크루스칼/프림)	★★★★☆	★★★	"최소 비용으로 모두 연결"
비트마스킹 / 비트마스크 DP	★★★★☆	★★★★★	" $n \leq 20$ 집합 상태", 외판원(TSP)
트리 알고리즘(트리DP·LCA)	★★★★☆	★★★★★	"트리에서 거리/조상/부분트리"
문자열(KMP·라빈카프·트라이)	★★★★☆	★★★★★	"패턴 매칭/접두사/부분문자열"
플로이드-워셜	★★★★☆	★★☆	"모든 쌍 최단경로(정점 작음 ≤ 500)"

Tier C — 심화 (고난도, 상위권 변별)

알고리즘	빈도	난이도	핵심 신호
세그먼트 트리 / 펜윅(BIT)	★★☆☆☆	★★★★★	"값 갱신 + 구간 질의가 둘 다 맞음"
벨만-포드 / SPFA	★★☆☆☆	★★★	"음수 간선/음수 사이클 탐지"
SCC(타잔/코사라주)	★☆☆☆☆	★★★★★	"방향그래프 강결합요소"
이분매칭 / 네트워크 플로우	★☆☆☆☆	★★★★★	"최대 매칭/최대 유량"
LCA·희소테이블·오일러투어	★☆☆☆☆	★★★★★	트리 고급 질의

추천 학습 순서 (ROI 기준)

1단계(합격선) 구현 → BFS/DFS → 정렬·이분탐색 → 그리디
 2단계(변별) DP(배낭·LIS·구간) → 투포인터/슬라이딩윈도우 → 누적합
 3단계(중급+) 다익스트라 → 유니온파인드 → 위상정렬 → MST → 비트마스크
 4단계(고득점) 세그먼트트리/펜윅 → 트리DP/LCA → 문자열(KMP/트라이) → 플로이드
 5단계(상위권) 벨만포드 → SCC → 네트워크플로우

시험·회사별 경향 (참고)

시험/회사	강하게 나오는 것
삼성 SW 역량테스트	구현/시뮬레이션 + BFS/DFS + 백트래킹(완전탐색) 비중 압도적
카카오(블라인드)	구현 + 자료구조 + 문자열 + 그리디/DP, "현실 문제" 서술형
라인/쿠방/배민 등	그래프·DP·이분탐색 골고루, 효율성(시간초과) 중시
대기업 일반 코테	Tier S 안정 + Tier A 일부면 통과선

마무리 — 코테 실전 루틴

1. **n** 보고 복잡도 목표 추정 → 가능한 알고리즘군 좁히기 (→ [자료구조.md](#) 부록2 A절).
2. **신호로 알고리즘 매칭**(위 표) → 가장 단순한 정답 후보부터.
3. **엣지케이스**: 빈 입력·최대 입력·중복·음수·오버플로(long)·1개 원소.
4. **시간초과 사**: 자료구조 교체(리스트→해시/힙), 전처리(누적합), 알고리즘 등급 상향(완탐→DP/이분).

코테 알고리즘도 본문의 실무 알고리즘과 **같은 사고**다 — 이 프로젝트의 **SKIP LOCKED** 작업분배(그리디+큐), 대사 매칭(해시 인덱스로 $O(n^2) \rightarrow O(n)$), 영업일 가산(시뮬레이션)처럼, *문제의 제약을 읽고 적합한 기법을 고르는 것*이 핵심.